Introduction to Machine Learning

Classification with Lazy Learning, SVM and RF

Ran Fan David Fernández Pol Marsol Mariam Shehadeh

October 25, 2026

Contents

1 Goals and Overview **2**

2 Datasets, EDA and Preprocessing **3**
2.1 Splice . . . 3
2.2 Vowel . . . 4
2.3 Preprocessing . . . 4

3 Base Models **5**
3.1 Random Forest . . . 5
3.2 Support Vector Machine . . . 5
3.3 k-Nearest Neighbors . . . 5

4 Reduction Techniques **7**

5 Methodology **9**
5.1 The 10 folds as blocks . . . 9
5.2 Metrics . . . 9
5.3 Selection rule and statistical tests . . . 9
5.4 Main effect and best configuration per level . . . 10
5.5 Selection bias . . . 10

6 Results **11**
6.1 kNN Results . . . 11
6.2 RF and SVM vs kNN Results . . . 12
6.3 Instance Reduction Results . . . 15

7 Discussion **17**
7.1 Feature selection or instance reduction? . . . 17
7.2 Combining feature selection and instance reduction . . . 17
7.3 Improving retention or reducing the training set? . . . 17

8 Conclusions and Limitations **19**
8.1 Conclusions . . . 19
8.2 Limitations . . . 19

1

GOALS AND OVERVIEW

The goal of this work is to compare a lazy learning classification model, a k-Nearest Neighbors classifier (kNN [1]), against two well studied eager learners: a Random Forest (RF [2]) and a Support Vector Machine (SVM [3]). The main difference is where each model spends most of the time, in training or in inference. The lazy learner stores the training cases (almost no time) and looks for the mos similar stored case in inference (where spends most of the time). In contrast, the eager learner builds a model first, forest of trees in RF or boundaries in SVM (where spends most of the time), and then only applies it in inference (almost no time).

We also want to study instance reduction: removing training cases before learning, to store less and answer faster, while trying to not increase error. We study three different types: condensation, edition and hybrid. Condensation type like the Fast Condensed Nearest Neighbor (FCNN [4]) aims to retain points which are closer to the decision boundaries. Edition type like the Edited Nearest Neighbor estimating Class Threshold (ENNTH [5]) seek to remove border points. Finally, the hybrid type like the DROP and DEL [6] allow the removal of internal and border points.

Each model is trained and tested on two different datasets, Splice and Vowel, each provided by the professor with 10 predefined train/test stratified folds. The Splice dataset has as instances DNA sequences of 60 nucleotides (nominal) and whether they represent an exon, an intron or neither. Vowel has as instances acoustic features (numerical) and has 11 different classes: the English vowels.

Particularly, for each model, we have checked several parameter combinations, comparing them and extracting the best ones to face them against the other model families. This comparison is done using statistical tests extracted from the Demšar paper [7].

All the data presented in this report, has been extracted from plots, reports and .csv that can be extracted from the project, following the README.md.

2

DATASETS, EDA AND PREPROCESSING

We use Splice (medium size, nominal) and Vowel (small, mostly numeric), each with the 10 predefined stratified train/test folds. Table 1 summarises them.

Dataset	Instances	Attributes	Classes	Class counts	Majority	Train / test
Splice	3190	60 nominal	3	EI 767, IE 768, N 1655	51.9%	2868–2873 / 317–322
Vowel	990	10 numeric + 3 nominal	11	90 per class	9.1%	891 / 99

Table 1: The two datasets. Train / test sizes are per fold. Neither dataset contains missing values.

2.1 SPLICE

Each instance is a window of 60 nucleotides, and the class says whether the centre of the window (between positions 30 and 31) is an exon–intron boundary (EI), an intron–exon boundary (IE) or neither (N). Besides A, C, G and T, the data contains four IUPAC ambiguity codes (D, N, S, R) in 60 of the 191 400 cells. We treat them as ordinary values, not as missing values.

The information is concentrated around the boundary. On the training part of fold 0, positions 28–35 carry 63% of the total mutual information between the positions and the class, the ten most informative positions carry 69%, and 24 of the 60 positions carry less than 0.01 bits. The canonical splice motifs are almost always present: 99.0% of EI instances have GT at positions 31–32, and 99.4% of IE instances have AG at positions 29–30 (Figure 1). Since our kNN gives the same weight to all 60 positions, we expect it to be at a disadvantage against SVM and RF on this dataset.

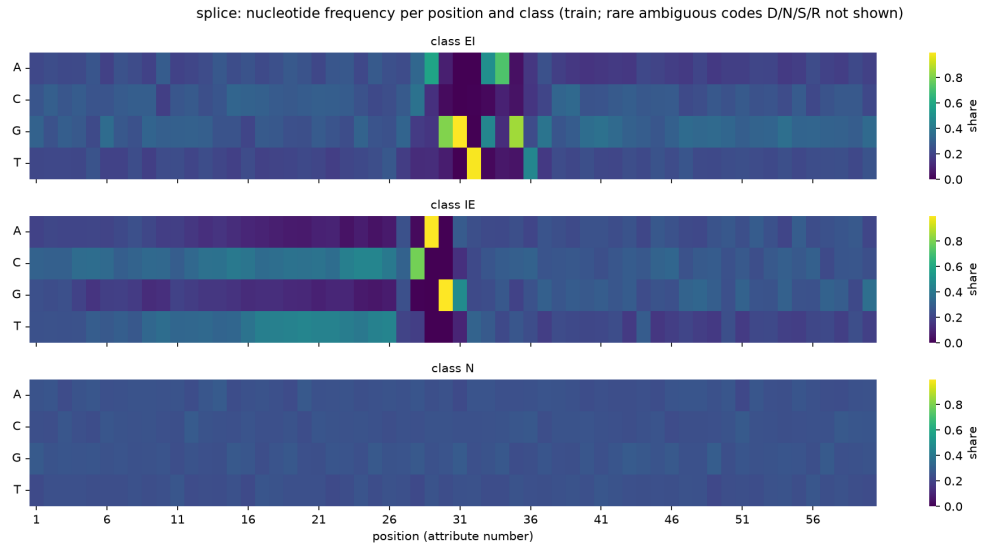


Figure 1: Share of each nucleotide per position and class in the training part of fold 0 of Splice. The largest differences are around the boundary, where EI shows GT at 31–32 and IE shows AG at 29–30. IE also has more C and T in the positions just before the boundary. Class N is close to uniform at every position.

Splice also contains duplicates. The 3190 rows contain 3005 distinct sequences (185 extra copies), and one sequence appears with two classes (IE and N). Each training fold has 5.0–5.6% extra copies, and 294 test rows (9.2% of all test rows over the 10 folds) have an identical row in their own training fold, always with the same class. We keep them. The assignment states that "*you cannot modify or perform a different cross-validation on the datasets*" (Section 1.1), and removing rows from a training or test

file would change the folds. The duplicates are also part of the data distribution the folds were built from. A test row with an exact copy in its training fold is found by the kNN at distance 0, and with inverse-distance voting only the exact matches then vote.

2.2 VOWEL

Vowel contains 15 speakers (8 male, 7 female), each saying 11 vowels 6 times ($15 \times 11 \times 6 = 990$). Each row has 10 numeric acoustic features and three nominal columns: `Train_or_Test`, `Speaker_Number` and `Sex`. There are no identical rows, but the folds are stratified by class, not grouped by speaker. In every fold, every (speaker, vowel) pair of the test part also appears in the training part, and all 15 speakers appear in every test fold. Using only the 10 numeric features, the nearest training case of a test case belongs to the same speaker in 972 of the 990 test cases (98.2%), and to the same speaker and vowel in 965 (97.5%). Consistent with this, a plain 1-NN (Euclidean, majority vote) already reaches 98.89% mean accuracy, against 76.33% on Splice. Vowel therefore measures how well a model recognises new repetitions by known speakers, not how it generalises to new speakers. We expect small differences between models on this dataset, and we keep this ceiling effect in mind when interpreting the results.

2.3 PREPROCESSING

All statistics are computed on the training fold only and then applied unchanged to the training and test parts, so no information from the test fold is used.

- **Numeric attributes** (Vowel) are min-max scaled to $[0, 1]$ with the minimum and maximum of the training fold, as the assignment requires. A test value outside the training range is clipped to 0 or 1. This keeps every distance term bounded and keeps all values in $[0, 1]$, and it affects only 21 of the 9900 numeric test cells (0.21%). A constant column would be set to 0. Missing values would be imputed with the training mean. The code handles them, although neither dataset contains any.
- **Nominal attributes** are stored as integer codes, given by the position of the value in the ARFF header. The codes are therefore identical in every fold, and a value that only appears in a test fold still has a valid code. Missing values would be imputed with the training mode. The codes are only labels: the kNN uses them to count mismatching attributes m (each mismatch adds 2 to the Euclidean, Manhattan, Canberra and Clark distances, as on a one-hot vector, and 1 to HEOM). SVM and RF receive a one-hot encoding instead (287 columns for Splice, 12 for Vowel). Given integer codes, both models would treat the values as ordered quantities (e.g. $A < C < G < T$), which has no meaning for nucleotides.
- **Vowel columns.** We drop `Train_or_Test`, which only records the half of the original UCI split that a row came from, and `Speaker_Number`, an identifier that carries no class information because every speaker says every vowel equally often. We keep `Sex`, which describes the voice. To check this choice, we ran the best Vowel kNN ($K = 3$, Euclidean, inverse-distance vote, always retain) with three column sets: (A) the 10 numeric features, (B) A + `Sex` (our choice) and (C) B + `Speaker_Number`. All three reach the same mean accuracy (0.9788). The per-fold accuracies differ by at most two test cases, Friedman gives $p = 1$, and the pairwise Wilcoxon tests give $p = 1$ (Holm-adjusted). The data therefore gives no reason to prefer another column set, so we keep B. Since the nearest neighbour is already the same speaker in 98.2% of the cases without `Speaker_Number`, adding the identifier changes little.

3

BASE MODELS

In this work, we have implemented three different classifiers, namely the Random Forest, the Support Vector Machine and the k-Nearest Neighbors, as presented in Section 1. We have implemented them following a common interface to abstract the models from the running code, thus the models can be used using the same run.py entry-point.

3.1 RANDOM FOREST

Random Forest [2] is an eager learner that builds an ensemble of `n_estimators` decision trees. Each tree is trained on a bootstrap sample of the training data. At every node split, the algorithm chooses a random subset of features determined by the parameter `max_features`.

We used the scikit-learn's `RandomForestClassifier` with single-core execution (`n_jobs=1`) to keep the execution time comparable with other classifiers. We explored the following hyperparameter grid using a customised grid_search:

- `n_estimators`: 50, 100, 200, 300. Number of trees in the ensemble.
- `criterion`: Gini vs Entropy. Function to measure split quality.
- `max_depth`: 10, 20, None (unlimited depth). Maximum depth of the trees.
- `max_features`: `'sqrt'` vs `None` (all features). The amount of features selected in each node split.
- `min_samples_leaf`: 1 vs 2. The minimum number of samples required to be a leaf.

This gives a total of 96 distinct Random Forest configurations per dataset.

3.2 SUPPORT VECTOR MACHINE

Support Vector Machine [3] is an eager learner that finds the optimal separating hyperplane with maximum margin between classes. The only instances that influence the final classification boundary, are the ones lying on or within the margin, called the support vectors. Nonlinear boundaries are constructed via kernel the kernel trick, using functions $K(x, x')$.

We explored three kernel families:

- **Linear kernel:** $K(x, x') = \langle x, x' \rangle$
- **Radial Basis Function (RBF) kernel:** $K(x, x') = \exp(-\gamma \|x - x'\|^2)$
- **Polynomial kernel:** $K(x, x') = (\gamma \langle x, x' \rangle + \text{coef0})^d$

We explored $C \in \{0.1, 1.0, 3.0, 10.0\}$, inversely proportional to the regularization of the model. $\gamma \in$ {'scale', 'auto'}, indicating the impact of the distance in the representation space, correlated to the variance of the model. Polynomial degrees $d \in \{2, 3, 5\}$ and coef0 $\in \{-0.5, 0.0, 0.5\}$, to customize the polynomial kernel. And `class_weight` $\in$ {'None', 'balanced'}, applying class weights when having an imbalanced dataset.

Grid Pruning: A naive grid search would produce 432 combinations. However, we would have several repeated models, as parameters like γ, d, and coef0 have no effect on a linear kernel, and d and coef0 have no effect on RBF, as previously mentioned. We pruned those identical models to avoid repeated runs. This left exactly 168 distinct SVM configurations (144 poly, 16 RBF, 8 linear). For multi-class classification, scikit-learn uses one-against-one (OAO) voting.

3.3 K-NEAREST NEIGHBORS

Unlike RF and SVM, our k-Nearest Neighbors classifier wasn't written using scikit-learn, instead, we only used NumPy. Being a lazy learner, training consists of simply storing the training examples in

memory ($O(1)$ time). However, in inference time, for each query q, the algorithm computes distances to all stored cases, retrieves the K closest neighbors and determines the predicted class through a voting policy.

Our implementation is fully vectorized. It calculates the full $N_{\text{test}} \times N_{\text{train}}$ distance matrix in NumPy. Neighbor distance ties are broken deterministically using a stable sort (`np.argsort(kind='stable')`), thus, the chosen one is the one with lower index in the matrix.

We compared models using grid search changing these four core parameters:

1. **Number of neighbors (K):** $K \in \{3, 5, 7\}$.
2. **Distance functions (δ):**
   - Euclidean (L_2): $d(q, x) = \sqrt{\sum_{\text{num}}(q_i - x_i)^2 + 2m}$
   - Manhattan (L_1): $d(q, x) = \sum_{\text{num}} |q_i - x_i| + 2m$
   - Canberra: $d(q, x) = \sum_{\text{num}} \frac{|q_i - x_i|}{|q_i| + |x_i|} + 2m$
   - Clark: $d(q, x) = \sum_{\text{num}} \left(\frac{|q_i - x_i|}{|q_i| + |x_i|}\right)^2 + 2m$
   - Heterogeneous Euclidean-Overlap Metric (HEOM): $d(q, x) = \sqrt{\sum_{\text{num}}(q_i - x_i)^2 + m}$

   where m is the number of mismatching nominal attributes.
3. **Voting schemes:**
   - *Majority voting:* Each neighbor gets 1 vote. Ties are broken by the class of the nearest neighbor among the tied classes.
   - *Inverse Distance Weighted (IDW):* Each neighbor votes with weight $w_i = 1/d(q, x_i)$. If a test sample is identical to a training sample ($d = 0$ and thus we would have division by 0), only exact matches vote with majority voting.
4. **Retention policies:**
   - *Never Retain (NR):* Standard kNN. The case base never changes during inference.
   - *Always Retain (AR):* Every query is appended to the case base with its true label.
   - *Different Class (DC):* An instance is only retained if it was classified incorrectly.
   - *Oblivion by Goodness (OG):* Every case c starts with goodness $G_0 = 0.5$. After classifying a query, retrieved neighbors receive reinforcement reward $r(c) = 1$ if their label matches the query, or 0 otherwise. Goodness is updated as $G_t(c) = G_{t-1}(c) + 0.2(r(c) - G_{t-1}(c))$. Cases whose goodness drops below $\epsilon = 0.5$ are permanently forgotten. We set $G_0 = \epsilon = 0.5$ to erase the bad cases early and keep decision in the middle of the Goodness range $[0, 1]$.

This produces $3 \times 5 \times 2 \times 4 = 120$ distinct kNN configurations.

4

REDUCTION TECHNIQUES

One of the major problems of kNN is the storage of the training cases. It produces a maintained memory demand and makes each query to check N_{train} distances . Furthermore, noisy or mislabled instances in the training set can decrease local classification correctness.

Instance reduction techniques try to solve this by selecting a smaller subset $S \subset T$ such that classifying with S maintains (or even improves) generalization accuracy. We implemented our own NumPy code for four reduction algorithms from the three classic families, selected as the most modern ones available for the assignment. Their taxonomy and versions evaluated can be visualised in Table 2.

Family	**Technique**	**Search Direction**	**Grid Variants Evaluated**
Condensation	FCNN [4]	Incremental	Version 1 (nearest enemy), Version 2 (medoid enemy)
Edition	ENNTH [5]	Decremental	Threshold $\mu \in \{0.0, 0.5, 0.7\}$
Hybrid	DROP3 [6]	Decremental	Version 3 (with noise filtering pass)
Hybrid	DEL [6]	Decremental	Cameron-Jones description length heuristic

Table 2: Taxonomy of instance reduction algorithms checked.

1. FCNN (Fast Condensed Nearest Neighbor): FCNN is an incremental condensation method designed to retain boundary prototypes while dropping redundant interior points. It starts with an empty prototype set S initialised with class medoids. In each iteration, it partitions remaining training samples into Voronoi cells around their nearest prototype in S. For each prototype, FCNN identifies training points in its Voronoi cell that are misclassified by S (using kNN), and adds the most representative enemy (Version 1 adds the nearest enemy, Version 2 adds the closest enemy medoid). FCNN is order-independent and terminates when S correctly classifies the entire training set. Versions 3 and 4, although implemented, were not considered for evaluation, as the only difference is the initialization of the prototype set to the medoid with predominant class.

2. ENNTH (Edited Nearest Neighbor Threshold): ENNTH is an edition technique that cleans out noise and smooths decision boundaries. The ENNTH looks at the K nearest neighbors of instance x and weights their votes by $1/(1 + d(x, x_j))$ to estimate the class probabilities $P(c|x)$, after applying normalization. An instance is edited out if its true label does not match the highest probability class, or if that maximum probability does not exceed the given threshold μ. With higher μ values, we edit samples in uncertain boundaries. We chose 0.0, 0.5 and 0.7 values, as significantly enough not to explode combinations in the grid_search. We didn't approach 1 further, as we didn't want a significant drop in accuracy due to too much editing.

3. DROP3 (Decremental Reduction Optimization Procedure 3): DROP3 is a hybrid method. It first applies an ENN noise-filtering pass to remove misclassified noise points. Then, it sorts all remaining instances by distance to their nearest enemy (furthest first), so cases deep in the interior of a class are evaluated before boundary cases. An instance P is removed if and only if its *associates* (the instances that have P as one of their $K + 1$ nearest neighbors) are classified (using kNN) at least as accurately without P as with P. We also implemented DROP1, DROP2, DROP4 and DROP5, but we selected DROP3 for evaluation as it was reported as having the best mix of storage reduction and generalization accuracy of the DROP methods in the paper [6].

4. DEL (Decremental Encoding Length): DEL uses the same search order as DROP3, but replaces the associates rule with a Minimum Description Length (MDL) criterion:

$$\text{Cost}(m, n, x) = F(m, n) + m \log_2 C + F(x, n - m) + x \log_2(C - 1) \tag{4.1}$$

where m is the number of prototypes kept, x is the number of training exceptions misclassified by S, and $F(m, n)$ is the combinatorial cost of encoding subset selection. A prototype is removed if the total

description length does not increase. We implemented DEL with log-sum-exp arithmetic to prevent floating-point overflow.

Cross-Model Sharing: To keep comparisons clean, the reduced prototype set S chosen by the best kNN is saved and passed directly to train SVM and Random Forest. This guarantees that all three model families are compared on the exact same training instances.

5

METHODOLOGY

We compare models and parameter settings following the procedure of Demšar [7].

5.1 THE 10 FOLDS AS BLOCKS

Each dataset comes with 10 fixed, stratified train/test folds that we may not modify. We treat the 10 folds as the blocks of every test ($N = 10$). Splice and Vowel are analysed separately and never merged or averaged together. Their error rates are not commensurable, and with only two datasets a test over datasets would have $N = 2$.

5.2 METRICS

For every configuration and fold we record the following quantities. Tables report their mean over the 10 folds.

- **Accuracy**: the fraction of correctly classified test instances. This is the metric used in every statistical test. We also report the standard deviation over the folds.
- **Macro F1**: the unweighted mean of the per-class F1 scores. We report it because Splice is imbalanced (51.9% of class N), and we look at it when accuracies are close.
- **Time**: the wall-clock time of the complete classifier call on one fold: fitting, predicting and, if used, the instance reduction. An eager model pays for its training here. We use it to compare model families.
- **Reduction time** and **solve time**. The reduction time is the time spent selecting the reduced training set. The solve time is the time minus the reduction time. We use the solve time to compare reduction techniques, because the reduction is a one-off cost and the question is whether the reduced set is faster to use.
- **Storage**: the percentage of the training set kept by the model before the test phase. It is 100% for every model without instance reduction, and the size of the reduced set for a reduced model.
- **Final case base**: the size of the kNN case base at the end of the test phase, as a percentage of the training set. We report it separately from storage because retention changes it while the test cases are solved. Always retain adds every solved case (111.1% on both datasets without reduction). Oblivion by goodness adds cases and also forgets them, so it can end below or above 100%. Never retain leaves the case base unchanged.

The grid search runs one configuration at a time, and the Random Forest uses `n_jobs=1`, so that all models are timed under the same single-core conditions as our kNN. All times come from a single run on one machine.

5.3 SELECTION RULE AND STATISTICAL TESTS

The winner of any group of configurations is chosen by a rule fixed before running the experiments: the highest mean accuracy over the 10 folds, then the lower mean time, then the configuration name. The tests never choose the winner. They only say whether it can be distinguished from the alternatives. We use non-parametric tests with $\alpha = 0.05$:

- **Two configurations**: Wilcoxon signed-ranks test with exact two-sided p-values over the 10 paired fold differences. The paired t-test is computed for reference only.
- **Three or more configurations**: Friedman test with the Iman–Davenport correction F_F, which is less conservative than the χ^2_F statistic.

- **Post-hoc tests**, only if the Friedman test rejects. We use Nemenyi for all pairs, with critical difference

$$\mathrm{CD} = q_\alpha \sqrt{\frac{k(k+1)}{6N}}, \tag{5.1}$$

  and a CD diagram for every significant Friedman test. When comparing several alternatives against a single control, we use two approaches depending on the objective:
  - **Bonferroni–Dunn** is used for the instance reduction comparisons ($k = 5$), as it provides a single critical difference threshold (CD) to display a confidence band around the unreduced control in the CD diagram.
  - **Wilcoxon signed-ranks with Holm's step-down adjustment** is used to test the control kNN against the eager challengers (kNN vs. SVM and kNN vs. RF). As it is noted in Demšar [7], Holm's step-down procedure is strictly more powerful than Bonferroni–Dunn while controlling the family-wise error rate at $\alpha = 0.05$, and also using the paired fold differences rather than discrete ranks.

With $N = 10$, the CD is 1.05 for $k = 3$, 1.48 for $k = 4$ and 3.32 for $k = 8$ (Nemenyi), and 1.77 for $k = 5$ (Bonferroni–Dunn).

We do not run a Friedman test over a whole grid (120 kNN or 168 SVM configurations). With $N = 10$, the CD would be far larger than any difference in mean rank. We test each factor separately and, for each model, its five best configurations.

5.4 MAIN EFFECT AND BEST CONFIGURATION PER LEVEL

For each hyperparameter (e.g. K, distance, kernel) we run two complementary tests:

- **Main effect**: per fold, the mean accuracy over all configurations that use a given level (value of the tested hyperparameter). No configuration is selected, so this test does not suffer from selection bias. It answers "is this level better on average?".
- **Best configuration per level**: the best configuration that uses each level. This is what "the best K" literally means, but it is selected on the same folds it is tested on, and a level with more configurations has more chances to win (the polynomial kernel has 144 SVM configurations, the linear kernel 8).

When both tests agree, the conclusion is solid. When they disagree, we report it: the level then helps only when it is tuned together with the other parameters.

5.5 SELECTION BIAS

Without a separate validation set, the winners are selected on the same 10 folds that are used to report and test them, so their accuracies are optimistic. To estimate this bias, we ran a leave-one-fold-out procedure (`experiments/selection_bias.py`). For each fold f, it picks the best configuration on the other nine folds (with the same selection rule) and scores it on f. Table 3 compares the mean of these ten held-out accuracies with the reported best mean.

The estimated bias is at most 0.004 accuracy points (0.4 percentage points). It is largest on Vowel, where several configurations tie or nearly tie: the Vowel SVM winner was chosen in only 3 of the 10 rounds. The bias is small compared with the differences between model families on Splice, but similar in size to some differences between configurations on Vowel. The tests that compare selected winners (best per level, top-5, model comparisons, reductions) therefore give optimistic p-values, and the main-effect tests are the ones free of this bias.

Dataset	Model	Configs	Reported	Held-out	Bias	Rounds with winner
Splice	kNN	120	0.8439	0.8439	+0.0000	9/10
Splice	SVM	168	0.9690	0.9680	+0.0009	8/10
Splice	RF	96	0.9690	0.9690	+0.0000	10/10
Vowel	kNN	120	0.9788	0.9747	+0.0040	7/10
Vowel	SVM	168	0.9909	0.9879	+0.0030	3/10
Vowel	RF	96	0.9798	0.9758	+0.0040	8/10

Table 3: Leave-one-fold-out estimate of the selection bias (base configurations, no reduction). "Rounds with winner": rounds in which the reported winner was chosen.

6

RESULTS

6.1 kNN RESULTS

Table 4 summarises the four kNN factors. For each level, "main" is the mean accuracy over all configurations with that level, and "best" is the accuracy of its best configuration (Section 5).

Data	Factor	Winner	Main effect: accuracy and test	Best per level: accuracy and test
Splice	K	$K = 7$	K=7 0.8292, K=5 0.8172, K=3 0.7978. Friedman $p = 6.9 \times 10^{-8}$; Nemenyi: K=7 ≈ K=5; K=3 worse than K=7.	0.8439, 0.8248, 0.8025. Friedman $p = 3.2 \times 10^{-7}$; same groups.
Splice	Distance	tie	All five 0.8147. Friedman $p = 0.991$.	All five 0.8439. Friedman $p = 1$.
Splice	Voting	IDW	IDW 0.8156, majority 0.8138. Wilcoxon $p = 0.0039$.	0.8439 vs 0.8411. Wilcoxon $p = 0.0625$ (n.s.).
Splice	Retention	OG	OG 0.8228, AR 0.8123, DC 0.8121, NR 0.8117. Friedman $p = 0.0011$; Nemenyi: OG ≈ AR.	OG 0.8439, DC 0.8269, NR 0.8266, AR 0.8254. Friedman $p = 0.025$; Nemenyi: OG ≈ DC ≈ NR.
Vowel	K	$K = 3$	K=3 0.9713, K=5 0.9538, K=7 0.9267. Friedman $p < 10^{-6}$; Nemenyi: K=3 ≈ K=5; K=7 worse than K=3.	0.9788, 0.9747, 0.9727. Friedman $p = 0.192$ (n.s.).
Vowel	Distance	n.s.	Manhattan 0.9535, Euclidean 0.9529, HEOM 0.9529, Canberra 0.9475, Clark 0.9462. Friedman $p = 0.266$.	Euclidean, Manhattan, HEOM 0.9788; Clark 0.9747; Canberra 0.9737. Friedman $p = 0.598$.
Vowel	Voting	IDW	IDW 0.9635, majority 0.9377. Wilcoxon $p = 0.0020$.	0.9788 vs 0.9747. Wilcoxon $p = 0.25$ (n.s.).
Vowel	Retention	AR	AR 0.9574, DC 0.9559, NR 0.9533, OG 0.9357. Friedman $p = 1.8 \times 10^{-7}$; Nemenyi: AR ≈ DC ≈ NR, OG worse than AR.	AR 0.9788, OG 0.9778, DC 0.9758, NR 0.9758. Friedman $p = 0.954$ (n.s.).

Table 4: kNN factor analysis (mean accuracy over the 10 folds). IDW: inverse-distance weighting; NR/AR/DC/OG: never retain, always retain, different class, oblivion by goodness. Nemenyi CD = 1.05 ($k = 3$) and 1.48 ($k = 4$).

On Splice, accuracy grows with K: $K = 7$ is significantly better than $K = 3$ in both tests, but not distinguishable from $K = 5$. The five distances rank the neighbours in the same order, and their mean accuracies are equal to four decimals. This is expected: all Splice attributes are nominal, so every distance is an increasing function of the number of mismatching positions m ($2m$, $\sqrt{2m}$ or $\sqrt{m}$), and the neighbours are the same. Inverse-distance voting is better on average (main effect), but the best configurations of the two voting schemes cannot be told apart. Oblivion by goodness has the highest accuracy in both tests, but it is not significantly better than every other policy: it ties with always retain in the main effect, and with different class and never retain in the best-per-level test. With oblivion by goodness, the final case base of the best configuration is 89.7% of the training set, so the

policy forgets more cases than it retains. A possible explanation is that it acts as an online editing step that removes misleading cases. We did not test this directly. The top 5 best configuration is a five-way tie, with $K = 7$, inverse distance voting and oblivion by goodness retention tie-break policies with the 5 different distances. The best one is with the Clark distance, only because of a minuscule wall-time difference.

On Vowel, accuracy falls with K: $K = 3$ beats $K = 7$ in the main effect, although the best configurations of the three values are not significantly different. In Section 2, the nearest training case of a test case was the same speaker and vowel in 97.5% of the cases. Our hypothesis is that larger K adds neighbours from other speakers. We did not measure this. No distance is significantly better. Inverse-distance voting is better on average, but again not when comparing the best configurations. Among the retention policies, oblivion by goodness has the lowest mean accuracy (0.9357) and is significantly worse than always retain. With $G_0 = 0.5$ and $\alpha = 0.2$, a single wrong retrieval (0.5 to 0.4) is enough to forget a case. For the best configurations, always retain (0.9788) and never retain (0.9758) are not significantly different (Friedman over the four policies, $p = 0.954$). Always retain ends with a final case base of 111.1% of the training set, never retain with 100%.

The best kNN configurations, chosen with the rule of Section 5, are $K = 7$, Clark, inverse-distance voting and oblivion by goodness on Splice (0.8439), and $K = 3$, Euclidean, inverse-distance voting and always retain on Vowel (0.9788). On both datasets, the top-5 configurations are not distinguishable from each other (Friedman $p = 1$ on Splice, $p = 0.986$ on Vowel), so other distances give the same result.

6.2 RF AND SVM VS KNN RESULTS

Table 5 summarises the factors of the two eager models.

Data	**Factor**	**Main effect: accuracy and test**	**Best per level: accuracy and test**
Splice	SVM kernel	RBF 0.9404, linear 0.9304, poly 0.7533. Friedman $p = 3.9 \times 10^{-10}$; Nemenyi: RBF $\approx$ linear; poly worse than RBF.	Poly 0.9690, RBF 0.9659, linear 0.9524. Friedman $p = 2.9 \times 10^{-7}$; Nemenyi: poly $\approx$ RBF; linear worse than poly.
Splice	SVM class weight	None 0.7855, balanced 0.7736. Wilcoxon $p = 0.037$.	0.9690 both. Wilcoxon $p = 1$.
Splice	RF criterion	Gini 0.9607, entropy 0.9606. Wilcoxon $p = 1$.	Entropy 0.9690, Gini 0.9671. Wilcoxon $p = 0.156$.
Splice	RF max_features	sqrt 0.9641, all 0.9572. Wilcoxon $p = 0.0039$.	sqrt 0.9690 vs all 0.9580. Wilcoxon $p = 0.0059$.
Splice	RF n_estimators	300: 0.9616, 200: 0.9615, 100: 0.9603, 50: 0.9591. Friedman $p = 4.3 \times 10^{-4}$; Nemenyi: 50 worse than 200; 300 and 100 not.	Friedman $p = 0.196$ (n.s.).
Vowel	SVM kernel	RBF 0.7290, poly 0.6881, linear 0.6614. Friedman $p = 3.9 \times 10^{-10}$; Nemenyi: poly and linear worse than RBF.	Poly 0.9909, RBF 0.9859, linear 0.7919. Friedman $p = 2.6 \times 10^{-6}$; Nemenyi: poly $\approx$ RBF; linear worse than poly.
Vowel	SVM class weight	0.6907 both. Wilcoxon $p = 1$.	0.9909 both. Wilcoxon $p = 1$.
Vowel	RF criterion	Entropy 0.9523, Gini 0.9421. Wilcoxon $p = 0.064$ (n.s.).	Entropy 0.9798, Gini 0.9758. Wilcoxon $p = 0.219$.
Vowel	RF max_features	sqrt 0.9673, all 0.9271. Wilcoxon $p = 0.0020$.	sqrt 0.9798 vs all 0.9475. Wilcoxon $p = 0.0020$.
Vowel	RF n_estimators	300: 0.9500, 200: 0.9491, 100: 0.9456, 50: 0.9440. Friedman $p = 0.046$; Nemenyi: no level differs from 300.	Friedman $p = 0.081$ (n.s.).

Table 5: SVM and RF factor analysis (mean accuracy over the 10 folds).

The best kernel depends on the test. On average, RBF is the most robust kernel, because many polynomial configurations perform poorly (poly has a main-effect accuracy of 0.7533 on Splice and

0.6881 on Vowel). When tuned, however, the polynomial kernel reaches the highest accuracy, and it cannot be distinguished from RBF on either dataset. The linear kernel is clearly worse on Vowel (0.7919 at best), which suggests that the Vowel classes are not linearly separable in the 12 input dimensions. Class weighting makes no difference on Vowel, which is balanced, and does not help on Splice: the main effect slightly but significantly favours no weighting. The RF criterion makes no significant difference on either dataset. Sampling $\sqrt{d}$ features per split (`max_features=sqrt`) is significantly better than using all features on both datasets, in both tests. On each dataset the five best SVM and the five best RF configurations are not distinguishable from each other (SVM: $p = 0.996$ and $p = 1$; RF: $p = 0.458$ and $p = 0.945$).

The parameters also change the time per fold (mean over the configurations of each level; single run, no test). For the kNN, the differences are small: on Splice the five distances take between 0.038 s (Canberra) and 0.052 s (HEOM), and on Vowel between 0.006 s (Euclidean) and 0.010 s (Clark). For the SVM, the polynomial kernel is the slowest on Splice (0.49 s against 0.38 s for RBF and 0.15 s for linear), and RBF on Vowel (0.016 s against 0.013 s and 0.011 s). For the RF, the time grows linearly with the number of trees (from 0.43 s with 50 trees to 2.46 s with 300 on Splice), and `max_features=sqrt` is 6.5 times faster than using all features on Splice (0.36 s against 2.34 s) and 2.5 times faster on Vowel (0.24 s against 0.58 s), on top of being more accurate.

Table 6 compares the best configuration of each family. As defined in Section 5, storage is the share of the training set kept by the model (100% for all of them, since none is reduced). The final case base is the size of the kNN case base after the test phase, which retention changes.

Data	**Model**	**Configuration**	**Accuracy**	**Macro F1**	**Time (s/fold)**	**Storage**	**Final case base**
Splice	kNN	$K = 7$, Clark, IDW, OG	0.8439 ± 0.0203	0.8407	0.040	100%	89.7%
Splice	SVM	poly, $d = 5$, $C = 0.1$, coef0 $= 0.5$, $\gamma =$ scale, balanced	0.9690 ± 0.0079	0.9650	0.607	100%	—
Splice	RF	entropy, 200 trees, no depth limit, sqrt, leaf 2	0.9690 ± 0.0100	0.9652	0.448	100%	—
Vowel	kNN	$K = 3$, Euclidean, IDW, AR	0.9788 ± 0.0154	0.9787	0.006	100%	111.1%
Vowel	SVM	poly, $d = 5$, $C = 1.0$, coef0 $= -0.5$, $\gamma =$ scale, no class weight	0.9909 ± 0.0088	0.9909	0.009	100%	—
Vowel	RF	entropy, 200 trees, depth 20, sqrt, leaf 1	0.9798 ± 0.0126	0.9797	0.345	100%	—

Table 6: Best configuration of each model family without instance reduction (mean ± standard deviation over the 10 folds). Time = fit + predict per fold.

On Splice, SVM and RF reach the same accuracy (0.9690) and are not significantly different (Nemenyi). Both are significantly more accurate than the best kNN, by 12.5 percentage points (Friedman $p = 3.8 \times 10^{-6}$; Wilcoxon with Holm correction $p = 0.0039$ for kNN vs SVM and for kNN vs RF). Our hypothesis, based on Section 2, is that the kNN gives the same weight to all 60 positions, while the signal is concentrated in a few positions around the boundary. RF and SVM can weight the positions differently. We did not measure which positions the eager models actually use.

On Vowel, the Friedman test rejects ($p = 0.032$), but the Nemenyi post-hoc test cannot separate any pair. The mean ranks are 1.35 (SVM), 2.30 (RF) and 2.35 (kNN), and the largest gap, 1.00, is below the CD of 1.05. The pairwise Wilcoxon tests agree (kNN vs SVM $p = 0.160$, kNN vs RF $p = 0.922$, Holm-adjusted). All three models lie between 97.9% and 99.1%. As discussed in Section 2, every test (speaker, vowel) pair is also in the training fold, which probably limits how much the models can differ.

In terms of time, the kNN has the lowest total time per fold on both datasets: 0.040 s against 0.448 s (RF) and 0.607 s (SVM) on Splice, and 0.006 s against 0.009 s (SVM) and 0.345 s (RF) on Vowel. The Friedman test on time rejects on both datasets. The kNN is not distinguishable from RF on Splice or from SVM on Vowel (Nemenyi).

Table 7 splits the time into fitting and prediction, normalised per training instance and per test instance. The kNN only stores the training set (below 10^{-10} s per instance), while SVM and RF spend between 9 and 380 μs per training instance. Per test instance, the eager models are faster than the kNN in all four cases, but by very different margins: SVM is 7.9 times faster on Vowel and RF 4.7 times faster on Splice, while SVM on Splice (111.5 against 124.8 μs) and RF on Vowel (58.1 against 60.1 μs) are almost as slow as the kNN. On these small datasets, the total time is dominated by fitting, which is why the kNN is the fastest model overall.

Data	**Model**	**Fit per training instance**	**Predict per test instance**
Splice	kNN	0.02 ns	124.8 μs
Splice	SVM	197.3 μs	111.5 μs
Splice	RF	151.5 μs	26.8 μs
Vowel	kNN	0.07 ns	60.1 μs
Vowel	SVM	8.8 μs	7.6 μs
Vowel	RF	380.5 μs	58.1 μs

Table 7: Fit and predict time of the best models, divided by the training and test fold sizes (mean over the 10 folds).

6.3 INSTANCE REDUCTION RESULTS

Each reduction technique was applied to the best kNN of each dataset. Following the selection rule of Section 5 (mean accuracy → mean per-fold time...), we extracted a champion for each variant: FCNN version 2, ENNTh with $\mu = 0$, DROP3 and DEL. The reduced training set was then given to the best SVM and the best RF. Table 8 shows the results. Each reduced model is compared with its unreduced version using Friedman and Bonferroni–Dunn against the control (CD = 1.77).

Data	Technique	kNN acc.	SVM acc.	RF acc.	Storage	kNN final case base	kNN solve time (s)	Reduction time (s)
Splice	None	0.8439	0.9690	0.9690	100%	89.7%	0.040	—
Splice	FCNN	0.8468	0.9596	0.9520	41.1%	30.1%	0.020	1.09
Splice	DROP3	0.8467	0.9135	0.9213	29.8%	23.2%	0.018	6.65
Splice	DEL	0.8464	0.9232	0.9106	42.9%	33.4%	0.024	10.22
Splice	ENNTh	0.8179	0.9637	0.9599	82.6%	71.8%	0.035	0.29
Vowel	None	0.9788	0.9909	0.9798	100%	111.1%	0.0059	—
Vowel	ENNTh	0.9515	0.9697	0.9535	97.0%	108.1%	0.0066	0.049
Vowel	FCNN	0.9414	0.9525	0.9061	29.6%	40.7%	0.0033	0.059
Vowel	DEL	0.9414	0.9313	0.9000	34.2%	45.3%	0.0037	0.74
Vowel	DROP3	0.9101	0.8889	0.8778	31.7%	42.8%	0.0040	0.58

Table 8: Instance reduction on the best model of each family (mean over the 10 folds). Storage: share of the training set kept by the reduction, the same for the three models. kNN final case base: size of the kNN case base after the test phase, which retention changes (OG on Splice, always retain on Vowel).

The following presented results, can be visualised in Figure 2.

On Splice, FCNN, DROP3 and DEL keep between 29.8% and 42.9,% of the training set and reach the same accuracy as the unreduced kNN (0.8464–0.8468 against 0.8439). The small increase is not significant. The three techniques also make the kNN significantly faster: the solve time falls from 0.040 s to 0.018–0.024 s per fold. ENNTh is the only technique that significantly lowers the accuracy (0.8179), while it keeps 82.6% of the data and does not significantly reduce the solve time. The selection rule picks FCNN as the best technique for Splice (highest accuracy), but FCNN, DROP3 and DEL cannot be told apart. DROP3 keeps the smallest set, and FCNN needs the shortest reduction time (total time − solve time) among the three (1.09 s against 6.65 s and 10.22 s).

On Vowel, FCNN, DEL and DROP3 significantly lower the accuracy (0.9101–0.9414 against 0.9788). ENNTh also lowers it (0.9515), but not significantly, and it is therefore the technique chosen for Vowel. It removes only 3% of the training set, so it gives almost no gain in storage or time. Only FCNN makes the kNN significantly faster on Vowel. In Section 2, the nearest training case of a test case was the same speaker and vowel in 97.5% of the cases. Our hypothesis is that condensation removes many of these repetitions, which are interior cases for the reduction but are exactly the neighbours that the kNN needs. We did not measure this.

When the reduced set of the kNN is given to SVM and RF, the eager models lose accuracy in every case, but the loss depends on the family of the technique. With ENNTh and FCNN the SVM is not significantly worse than the unreduced SVM on either dataset (e.g. 0.9596 against 0.9690 with FCNN on Splice). With DROP3 and DEL, the hybrid techniques, both SVM and RF are significantly worse on both datasets (down to 0.9106–0.9232 on Splice and 0.8778–0.9313 on Vowel). RF is also significantly worse with FCNN on Vowel (0.9061). In exchange, the reduced models are faster: on both datasets, SVM and RF trained on DROP3 or DEL sets have a significantly lower solve time.

Finally, comparing the four retention policies and the four reductions of the best kNN in one Friedman test (8 variants, Nemenyi CD = 3.32), on Splice only ENNTh is distinguishably worse from the best variant (OG without reduction). On Vowel, FCNN, DROP3 and DEL are significantly worse than always retain without reduction, while ENNTh and the other three retention policies are not.

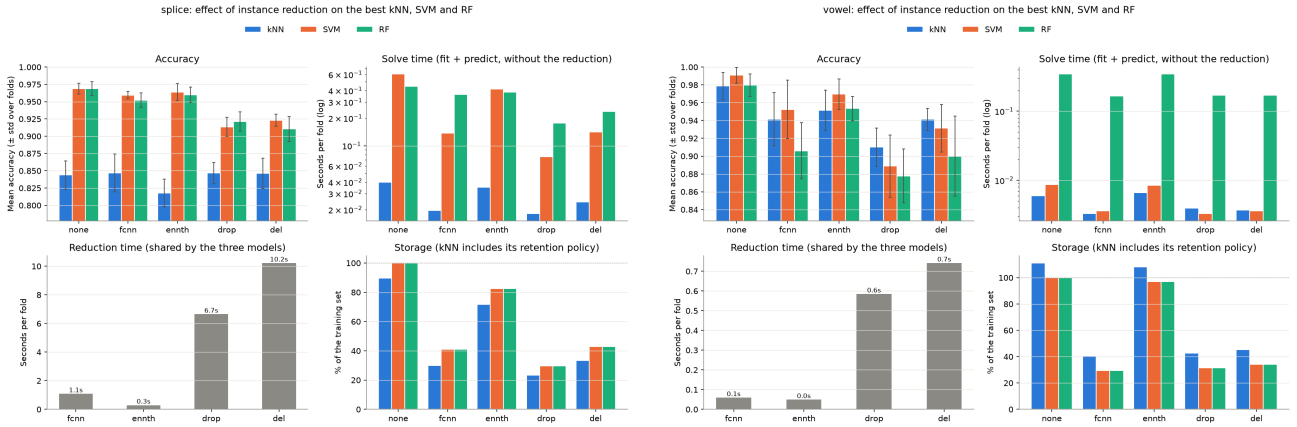


Figure 2: Accuracy, solve time, reduction time and storage of the best kNN, SVM and RF with each reduction technique (Splice left, Vowel right). In the storage panel, the kNN bar shows the final case base after retention, while the SVM and RF bars show the storage defined in Section 5.

7

DISCUSSION

The assignment asks three questions about feature selection, instance reduction and retention. We did not implement feature selection, so our answers to the first two questions combine our results with arguments that we have not tested.

7.1 FEATURE SELECTION OR INSTANCE REDUCTION?

Do you think it would be much better to perform feature selection rather than reducing the training set with a sample reduction technique?

The two methods change different things. Instance reduction removes rows: it saves memory and time and can remove noisy cases, but the distance between two cases stays the same. Feature selection removes columns, so it changes the distance itself and therefore which cases are neighbours.

On Splice, our results point to feature selection as the more useful of the two. The class information is concentrated in a few positions: positions 28–35 carry 63% of the mutual information and 24 of the 60 positions carry less than 0.01 bits (Section 2). Our kNN gives the same weight to all 60 positions and reaches 0.8439, against 0.9690 for SVM and RF. Instance reduction did not close this gap: FCNN, DROP3 and DEL keep the same accuracy as the unreduced kNN (0.8464–0.8468, not significantly different) with 29.8–42.9% of the training set. They make the kNN smaller and faster, but not more accurate. We therefore expect that removing or down-weighting the uninformative positions would help the kNN on Splice more than any reduction, but we did not run this experiment.

On Vowel, the kNN already reaches 0.9788 and cannot be distinguished from SVM and RF, so there is little room for improvement from either method. Here every reduction except ENNTh significantly lowers the accuracy.

7.2 COMBINING FEATURE SELECTION AND INSTANCE REDUCTION

Do you think that the combination of both, feature selection and sample reduction, will be the best option to obtain the best kNN algorithm?

We think so, especially on Splice, where each method addresses a different weakness: feature selection the accuracy and reduction the storage and the time. The order matters. The reduction techniques decide which cases to keep from their neighbours or them being neighbours (FCNN keeps the cases needed to classify the rest, ENNTh keeps the cases correctly classified by their neighbors, and DROP3 and DEL check the check if they are key to classify the cases from which they are neighbors). If these neighbours are computed with uninformative positions, the techniques may keep or remove the wrong cases. Feature selection should therefore come first, and the reduction should be run on the selected features. Both steps would have to be fitted on the training fold only, and choosing their parameters on the same 10 folds would add to the selection bias discussed in Section 5. This answer is a hypothesis: we did not test the combination.

7.3 IMPROVING RETENTION OR REDUCING THE TRAINING SET?

Do you think it will be much better to improve the retention process rather than reducing the training set?

Our results answer this question differently on the two datasets. On Splice, the best retention policy, oblivion by goodness, reaches 0.8439 and ends with 89.7% of the training set. FCNN, applied before the same kNN, reaches 0.8468 with 41.1% of the training set (30.1% at the end of the test phase) and halves the solve time (0.020 s against 0.040 s). In a Friedman test over the four retention policies and

the four reductions, only ENNTh is distinguishable from the best variant. Reduction therefore gives the same accuracy with a much smaller case base, so on Splice reducing the training set is the better choice.

On Vowel, the four retention policies cannot be told apart (best configurations, $p = 0.954$), while FCNN, DROP3 and DEL are significantly worse than always retain. Neither method improves the kNN here, and reduction costs accuracy.

The two methods are also different in nature. Retention runs during the test phase, so its result depends on the order of the test cases. Reduction runs once on the training set, before any test case is seen. They are not exclusive: in our experiments, each reduced kNN still uses its retention policy.

8

CONCLUSIONS AND LIMITATIONS

8.1 CONCLUSIONS

The best kNN on Splice uses $K = 7$, inverse-distance voting and oblivion by goodness (0.8439). The five distances rank the neighbours identically there, because all attributes are nominal. On Vowel, the best kNN uses $K = 3$, the Euclidean distance (tied with Manhattan and HEOM), inverse-distance voting and always retain (0.9788). The best configurations of the four retention policies are not significantly different on Vowel, so never retain (0.9758) is an equally valid choice with a smaller case base.

For SVM, the polynomial kernel of degree 5 gives the best model on both datasets (0.9690 on Splice, 0.9909 on Vowel). It cannot be distinguished from the best RBF model, and RBF is the better kernel on average. For RF, the split criterion makes no significant difference, while sampling $\sqrt{d}$ features per split is significantly better than using all features. The best RF reaches 0.9690 on Splice and 0.9798 on Vowel.

Comparing the two paradigms, SVM and RF are significantly more accurate than the kNN on Splice, by 12.5 percentage points, while on Vowel no pair of models can be distinguished. The kNN has the lowest total time per fold on both datasets, since its training only stores the data. Per test instance, the eager models predict faster than the kNN, clearly for RF on Splice and SVM on Vowel, but only marginally for SVM on Splice and RF on Vowel.

Instance reduction is useful for the kNN on Splice only. There, FCNN, DROP3 and DEL keep the accuracy of the unreduced kNN with 29.8–42.9% of the training set and about half the solve time. The selection rule picks FCNN, which is also the fastest of the three to compute, but the three cannot be told apart. On Vowel, ENNTh is the only technique that does not significantly lower the accuracy, and it removes only 3% of the cases.

When the reduced sets are given to the eager models, the SVM trained on the ENNTh or FCNN sets is not significantly worse than the unreduced SVM. With the hybrid techniques, DROP3 and DEL, both SVM and RF are significantly worse on both datasets, although they become faster.

8.2 LIMITATIONS

Our tests use the 10 folds as blocks, but any two training folds share most of their instances, so the blocks are not independent and the p-values are approximate [7]. The best configurations are also selected and tested on the same folds. Our leave-one-fold-out estimate of this selection bias is at most 0.004, but the tests that compare selected winners give optimistic p-values.

The data limit what the results can show. The Vowel folds are not split by speaker, so every (speaker, vowel) pair of a test fold also appears in its training fold. This probably explains the high accuracy of all models and hides differences between them. On Splice, 9.2% of the test rows have an identical copy in their training fold. In both cases a fix would change the folds, which the assignment does not allow.

All times come from a single run on one machine, with one configuration at a time and `n_jobs=1` for RF. They are useful to compare the models, but small differences should not be over-interpreted. Finally, several explanations in this report, such as why a larger K hurts on Vowel, why oblivion by goodness helps on Splice, why reduction hurts on Vowel and how much feature selection would help, are hypotheses that we did not measure.

REFERENCES

[1] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, Jan. 1967.

[2] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.

[3] C. J. Burges, "A Tutorial on Support Vector Machines for Pattern Recognition," *Data Mining and Knowledge Discovery*, vol. 2, pp. 121–167, 1998.

[4] F. Angiulli, "Fast Nearest Neighbor Condensation for Large Data Sets Classification," *IEEE Transactions on Knowledge and Data Engineering*, vol. 19, no. 11, pp. 1450–1464, Nov. 2007.

[5] F. Vázquez, J. S. Sánchez, and F. Pla, "A Stochastic Approach to Wilson's Editing Algorithm," in *Pattern Recognition and Image Analysis (IbPRIA 2005)*, Lecture Notes in Computer Science, vol. 3523, pp. 35–42, 2005.

[6] D. R. Wilson and T. R. Martinez, "Reduction Techniques for Instance-Based Learning Algorithms," *Machine Learning*, vol. 38, no. 3, pp. 257–286, Mar. 2000.

[7] J. Demšar, "Statistical Comparisons of Classifiers over Multiple Data Sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, Dec. 2006.

[8] D. R. Wilson and T. R. Martinez, "Improved Heterogeneous Distance Functions," *Journal of Artificial Intelligence Research*, vol. 6, pp. 1–34, 1997.

[9] D. W. Aha, D. Kibler, and M. K. Albert, "Instance-based learning algorithms," *Machine Learning*, vol. 6, no. 1, pp. 37–66, 1991.

[10] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Proc. 14th Int. Joint Conf. on Artificial Intelligence (IJCAI)*, vol. 2, pp. 1137–1145, 1995.